CODE GUIDE

How the pipeline works, and why it is built this way

What each part does, what was used, and what it can defend

DM-AHR-ViT · Explainable Vision Transformer for Skin Lesion Classification Using Dermoscopy Images

Research prototype only. Not clinically validated and not intended for diagnosis, triage or patient care.

How to read this document

Sections 1–3 orient you: the study in a page, what was actually used, and the repository map. Section 4 walks each pipeline phase with its reason for existing. Sections 5–7 are the audit trail: the source of every reported number, each design decision, and every deviation from the proposal. Sections 8–10 prepare you for questions: a glossary, likely supervisor questions with grounded answers, and an explicit list of what is not claimed.

1. The study in one page

Question. Dermoscopy images contain hair that obscures lesion structure. Does removing it with a generative (diffusion) model help a Vision Transformer classify skin lesions better than classical image-processing hair removal, or than leaving the hair in?

Design. Four input conditions, everything else held identical:

Arm	Input the classifier sees	Purpose
A	original image	baseline: no hair removal
B	DullRazor-style morphological detection + Telea inpainting	classical hair removal
C	first diffusion run (defective trainer; recorded failure)	generative hair removal, failed
Cv2	rebuilt diffusion hair removal (Phase 14)	generative hair removal
A∗	original, bicubic 128 px round-trip	control for Arm C's resolution change

Data. HAM10000, 7 diagnostic classes, split 70/15/15 **by lesion** so one lesion never appears in two splits. Three seeds (42, 123, 2026) per arm.

Three hypotheses, stated before results.

- H1 (image quality): diffusion removal preserves lesion texture better than classical inpainting.
- H2 (classification): the classifier does better on diffusion-cleaned images.
- H3 (explainability): Grad-CAM and attention maps focus more on the lesion after diffusion removal.

What "success" means. A null or negative result, honestly reported, is a successful outcome. Nothing here promises that diffusion wins.

2. The data

What the corpus is

HAM10000 is a public dermoscopy collection: **10,015 images** of skin lesions, each with one of seven diagnoses, published through the ISIC Archive. The client's collection (ISIC collection 212) exposes **11,720** images — the 10,015 labelled ones plus 1,705 that carry no diagnosis in this corpus. Only the labelled set is used; the extras are quarantined and cannot enter a split.

Verified properties, recomputed rather than assumed

Property	Value	Why it matters
Images in collection 212	11,720	the download is complete, paged to exhaustion
Labelled HAM10000 cohort	10,015	every reported metric comes from this set
Quarantined extras	1,705	unlabelled; excluded from every split
Pixel dimensions	600 × 450, every image	uniform geometry, so no per-image resize rules needed
Colour mode / encoding	RGB / JPEG	no greyscale or palette images to special-case

Property	Value	Why it matters
Decode status	11,720 / 11,720 ok	no corrupt or truncated files
Median file size	~27 KB	consistent with lossy JPEG at this resolution
Lesion groups	7,470	the unit of splitting and of every statistic
Diagnoses	7 classes	akiec, bcc, bkl, df, mel, nv, vasc

The imbalance problem

`nv` (melanocytic naevus) is roughly two thirds of the corpus; `df` and `vasc` number in the tens. This single fact dictates several later choices: macro-F1 rather than accuracy as the primary endpoint, class-weighted loss, and a stratified split so the rare classes keep a non-zero test presence. It also means per-class numbers for `df` (17 test images) and `vasc` (21) are noisy and must not be read individually.

The leakage problem

HAM10000 contains several photographs of the same lesion, taken in different sessions. If those photographs were scattered across training and test, the classifier would be scored on lesions it had already seen, and the result would measure memorisation. The corpus therefore has to be split by **lesion**, not by image. A dHash sweep proposed 70,062 candidate duplicate pairs; pixel confirmation on a common 96×96 grid reduced that to 21 genuine pairs — and all 21 already shared a lesion identifier, so grouping equals the metadata's own lesion field (7,470 groups). The check's value is that it rules out *unlinked* duplicates, not that it changes the grouping.

Reference material the study also needs

Resource	Status	Use
HAM10000 lesion masks	available, 10,015	reference for explanation overlap
Released DM-AHR weights	available (Google Drive)	smoke test only; training data undocumented
Synthetic hairy/hairless pairs	generated here, 1,022	the only source of a true hairless reference
DERMAHAIR paired data	blocked (Kaggle-gated)	would have given real paired references
Lama et al. hair masks	not obtained	the proposal's independent screening reference

The last two rows are gaps, not findings. They are recorded as skipped steps in the decision log, not described as properties of the data.

3. Methodology

The question and the three hypotheses

The study asks whether **generative (diffusion) hair removal** helps a skin-lesion classifier more than **classical hair removal**, or than leaving the image alone. Three hypotheses were stated before any result existed:

- H1 (image quality): diffusion removal preserves lesion boundaries and texture better than

classical inpainting. Measured by PSNR and SSIM against a true hairless reference.

- H2 (classification): a ViT trained on diffusion-cleaned images outperforms the original and

classically cleaned arms. Measured by macro-F1 on the locked test set.

- H3 (explainability): Grad-CAM and attention maps focus more on the lesion after diffusion

removal. Measured by lesion overlap and a pointing game against the released masks.

These are hypotheses, not expectations. A null or negative result, honestly reported, is a successful outcome.

The experimental design

Four input conditions, everything else held identical:

Arm	Input the classifier sees	What it isolates
A	the original image	baseline: no hair removal
B	DullRazor-style morphological detection + Telea inpainting	classical hair removal
C	first diffusion run (defective trainer; recorded failure)	generative hair removal, failed
Cv2	rebuilt diffusion hair removal (Phase 14)	generative hair removal
A∗	the original passed through a 128 px round-trip	Arm C's resolution change, alone

Arm A∗ exists because DM-AHR works natively at 128 px while the ViT expects 224. Without it, a drop in Arm C could be blamed on diffusion artefacts when the resolution round-trip alone caused it. A∗ is the control that makes that separable.

Why each design choice was made

Choice	Reason
Split by lesion, frozen and hashed before training	a split chosen after seeing results cannot be trusted
Cohort limited to the 10,015 labelled images	the extras have no ground truth to score against
Hair threshold frozen before test scoring	otherwise the detector is tuned on the test set
Diffusion model trained here, on train/val sources	the released weights have undocumented training data, so leakage cannot be excluded
Test set scored once per frozen checkpoint	repeated scoring turns the test set into a tuning set
Three seeds (42, 123, 2026)	reduces the chance that one lucky initialisation drives the comparison
Identical recipe across arms	the hair-removal condition must be the only difference

How the results are judged

Two tests per endpoint, as the protocol specifies. A **lesion-group bootstrap** resamples whole lesions 10,000 times, sharing the same resample across arms and seeds, giving a percentile 95% confidence interval. A **Wilcoxon signed-rank** test is run on values collapsed to one per lesion group, so the several photographs of one lesion are not counted as independent evidence. Holm correction is applied across the three planned

comparisons within each endpoint, separately for each test. A confidence interval spanning zero is called **inconclusive**, not "no difference".

Effect size matters as much as the p-value: with 1,437 test groups a trivial difference can reach significance. Results whose standardised effect is negligible are flagged as such.

4. The ML pipeline, phase by phase

The pipeline has thirteen phases. Each one reads files, writes artifacts and logs what it did, so any later number can be recomputed from disk. Phases 1–3 and 12 are CPU work; 4–11 need a GPU.

Three properties hold throughout. Nothing is skipped silently — every excluded image is counted and written to a CSV with a reason. Long jobs are resumable, so a preempted container continues rather than restarting. And no phase may consume a later phase's output, which is what keeps the test set out of every design decision.

Each phase below states **what it does**, **why it is built that way**, and **what it writes**. The artifact is the evidence; a phase that writes nothing did not happen.

Phase 1 — Acquisition (00_download_data.py)

Fetches all 11,720 images of ISIC collection 212 and records size, checksum, dimensions, mode and decode status for each.

Why: the client's collection has more images than the paper's 10,015. Downloading the collection as shipped, and then deciding explicitly what belongs to the confirmatory cohort, is safer than guessing which subset was intended.

Writes: `data/manifests/data_manifest.csv`, `artifacts/logs/data_audit.json`.

Phase 2 — Audit, deduplication and the frozen split (01_audit_and_splits.py)

Three things happen here, in order:

1. **Cohort definition.** The 10,015 images matching HAM10000 metadata are the confirmatory cohort (C0). The remaining 1,705 are quarantined as `C0-extra` and can never enter a split. 2. **Near-duplicate merging.** dHash finds candidate pairs cheaply; every candidate is then re-checked on a common 96×96 greyscale grid and kept only if mean absolute difference ≤ 4/255 and Pearson r ≥ 0.985. Groups are `lesion_id` merged with those confirmed pairs. 3. **Splitting.** 70/15/15, apportioned per class under a global size cap, with groups never split across partitions. The split table is hashed and frozen.

Why this matters more than any model choice: if one lesion appears in both train and test, the test score measures memorisation, not generalisation. dHash alone proposed 70,062 candidate pairs — far too many to be real duplicates, because similar dermoscopic backgrounds collide. Pixel confirmation reduced that to 21 genuine pairs.

Writes: `data/manifests/splits.csv`, `split_summary.json`, `data_audit.json`.

Result: 10,015 images → 7,470 lesion groups → 7,011 / 1,502 / 1,502 images, zero groups spanning two splits.

Phase 3 — Hair screening, Arm B and synthetic pairs (02_hair_pipeline.py)

Screening. A multi-orientation black-hat/top-hat morphological detector estimates hair coverage. Images above the frozen threshold are hair-present.

Arm B. Detected hair is removed with morphological closing plus Telea inpainting.

Synthetic pairs. For hair-*absent* images, realistic hair is composited with a recorded seed, giving a hairy input and a genuine hairless reference. This is what makes PSNR/SSIM possible at all.

Why the threshold is frozen here: it is tuned on train/validation synthetic pairs and fixed before any test image is processed. A parameter adjusted after seeing test behaviour would invalidate the test set.

Writes: `hair_screening.csv`, `processed/B/`, `synth/`, `hair_pipeline_summary.json`.

Result: 8,993 of 10,015 images (89.8%) screened hair-present; 1,022 pairs, of which 186 are test.

Phase 4 — DM-AHR audit and smoke test (03_dmahr_smoke.py)

Loads the pinned upstream code, parses its config, loads the released checkpoint and runs a batch.

Why an audit before training: the upstream code has two defects that must be understood rather than papered over. Its sampler returns only the last image of a batch, and its diffusion schedule is never initialised. Both are worked around explicitly, with the failure documented in the decision log, rather than by substituting a different model.

Writes: `dmahr_smoke.json`, panel image.

Phase 5 — Diffusion training (04_dmahr_train.py)

Trains DM-AHR on **train-split sources only**, selecting on validation.

Why not use the released weights: the DM-AHR paper says they were trained on HAM10000 images other than a 700-image test set, so they very probably saw our validation and test images. Using them could leak test images into Arm C. The released weights are used for the smoke test only, never for Arm C.

Deviation: 20,000 iterations against 660,000 in the upstream config, and a training script that gave the network an inverted noise-level signal (found after the primary results; see `reports/oneshot_diagnostic_report.md`). Together they are the single most important caveat on Arm C.

Writes: `artifacts/checkpoints/dmahr/`, `dmahr_train.json`, `training_manifest.json`.

Phase 6 — Arm C generation (05_dmahr_sample.py)

Samples hair-removal output for every hair-present image, sharded across containers, skipping already-written outputs so a preempted run resumes.

Deviation: 1,000 sampling steps instead of the 2,000 reference, with bf16 autocast, forced by the account balance. Every per-image row records `num_timesteps`, so no reader can mistake the ablation for the full method.

Writes: `processed/C/`, per-shard CSVs.

Phase 7 — Arm C passthrough (05b_fill_c_passthrough.py)

Copies the original image into `processed/C` for hair-absent images.

Why: those images are byte-identical in every arm, because there is no hair to remove. Without this step Arm C's dataset is incomplete and training fails.

Phase 8 — ViT training (06_train_vit.py)

One run per (arm, seed): 12 runs. ImageNet-pretrained ViT-B/16, AdamW at 3e-5, warm-up then cosine decay, batch 32, inverse-frequency class weights from the **train split only**, early stopping on validation macro-F1, gradient clipping, bf16.

Why class weights rather than balanced sampling: the dev kit makes weighted loss primary and warns against combining the two; the proposal's looser wording mentions both. Recorded as D16.

Deviation: augmentation is rotation ±15° with a resized crop, not the proposal's ±90° with explicit zoom/shift. It is identical across arms, so the comparison stays internally valid; recorded as D15.

Writes: `artifacts/checkpoints/vit/<arm>_seed<seed>/` with config, seed, class weights, git commit.

Phase 9 — Evaluation (07_eval_vit.py)

Scores each frozen checkpoint on the locked test set once. Emits per-image predictions, not just aggregates, so every downstream statistic can be recomputed.

Writes: `artifacts/predictions/pred_<arm>_seed<seed>_<subset>.csv`, `classification_metrics_all.csv`, `classification_per_class.csv`.

Phase 10 — Image quality (07b_image_quality.py)

PSNR and SSIM for Arm B and Arm C against the synthetic hairless reference, with the *unprocessed hairy input* scored as a floor.

Why the floor matters: a method that scores below the image it was given has actively damaged it. Arm C scored below the hairy input, which is how the failure was first detected. A follow-up diagnosis (`scripts/17_arm_c_diagnose.py`, written up in `reports/arm_c_diagnosis.md`) first concluded the checkpoint was undertrained. That conclusion is retracted: the training script fed the network an inverted noise-level signal, found later and documented in `reports/oneshot_diagnostic_report.md`. The released DM-AHR checkpoint reaches PSNR 26.8 / SSIM 0.684 through the same inference path, while the checkpoint trained here reaches 15.0 / 0.079.

Phase 11 — Explainability (08_xai.py)

Grad-CAM on the patch-embedding grid and attention roll-out, both compared with the lesion mask. Two measures: the fraction of the top-20% salient pixels inside the mask, and a pointing game on the single most salient pixel. Uniform and centred-Gaussian chance baselines are computed per image.

Why chance baselines: masks vary in size, so a large lesion inflates overlap for any saliency map. Overlap without a chance baseline is not interpretable.

Writes: `xai_localization_<arm>_seed<seed>_X1.csv`, `xai_summary_*.json`, overlay panels.

Phase 12 — Statistics (09_stats.py)

Two tests per endpoint, as the protocol requires:

- Lesion-group bootstrap: resample whole lesions 10,000 times, the same resample shared across

arms and seeds, percentile 95% CI.

- Wilcoxon signed-rank: per-lesion values collapsed to one number per group before testing.

Holm correction is applied across the three planned pairwise comparisons within each endpoint, per test. Cohen's dz is computed on observed pairs, not the bootstrap distribution (whose spread shrinks with replicate count and would inflate the effect size).

Why groups and not images: a lesion with six photographs would otherwise contribute six independent-looking data points. Collapsing to the lesion is what makes the p-value honest.

Writes: `statistical_comparisons.csv` (20 comparisons across 5 endpoints).

Phase 13 — Report and gates (10_finalize.py, check_acceptance.py)

The report is generated from the artifacts, not typed by hand, so no number in it can drift from its source. The inventory records a SHA-256 for every deliverable, and the gate checker re-verifies the acceptance criteria.

Current state: 14 of 14 gates pass. One functional item, F04 (DERMAHAIR source pairs), is honestly marked **PENDING** because the dataset is Kaggle-gated and no credentials were available.

Phase 14 — Rebuilt diffusion arm Cv2 (Stages A, B and C)

Arm C above was made with a trainer that gave the network an inverted noise-level signal; it stays in the record as a failed run. The rebuilt arm Cv2 is documented in `reports/prereg_v2_diffusion.md` (plan, results, deviations) and `reports/protocol_addendum_stageC.md` (what was frozen before the test split was used). In code:

- src/dm_ahr_vit/diffusion/train.py mirrors upstream's loss and noise-level convention; contract

tests are in `tests/test_noise_level_convention.py` and `tests/test_trainer_sampler_contract.py`.

- respaced_super_resolution(..., start_t=600) in src/dm_ahr_vit/diffusion/dmahr_adapter.py starts

the strided DDIM chain from the noised input instead of pure noise (Stage A; `scripts/38_partial_noise_eval.py`).

- scripts/40_harvest_hair.py and src/dm_ahr_vit/hair/harvest.py cut thin, long, dark hair strands

out of train and validation photos (never test). The frozen screening config (`configs/hair_screen.json`, hash-checked by `load_hair_screen_config`) supplies the detector. `scripts/43_harvest_review.py` writes the contact sheets used to review the harvest.

- src/dm_ahr_vit/data/hair_composite.py pastes harvested strands onto clean train photos on the fly;

`scripts/04_dmahr_train.py --composite --warm-start <ckpt>` fine-tunes on them.

- Validation-only checks: scripts/37_real_hair_check.py (detector ratio on real photos),

`scripts/45_composite_eval.py` (ground truth on pasted strands), `scripts/46_dense_hair_sheet.py`.

- Stage C: scripts/05_dmahr_sample.py --arm-name Cv2 --respaced --start-t 600, `05b_fill_c_passthrough.py

--arm-name Cv2, then 06/07/08 with --arm Cv2, 07b_image_quality.py --c-arm Cv2 and 09_stats.py --c-arm Cv2 (outputs carry a _Cv2 suffix). Modal drivers are in modal_app.py (cv2_sample_driver, vit_arm_driver, stats_arm`).

Honest status: the pre-registered real-photo gate for Stage B was not met (decision D34) and Stage C was run anyway as a recorded deviation; read `reports/stageC_results.md` for what that does and does not show.

Later stages (all documented in `reports/`):

- Stage D (reports/prereg_v3_pasteback.md, scripts/48_pasteback.py, src/dm_ahr_vit/hair/pasteback.py): full-resolution

paste-back of the diffusion fill. Failed its validation gate (SSIM 0.846 against 0.90); never used on test data.

- Stage E (reports/prereg_v4_review_response.md, reports/stageE_results.md): response to an independent review.

`scripts/53_pooled_analysis.py` (with `src/dm_ahr_vit/stats/pooled.py`) pools eight seeds with a two-level bootstrap; `50_composite_arms.py`, `51_condition_usage.py`, `52_residual_hair_split.py` are validation-only checks; `54_review_kit.py` builds and analyses a blinded human-review kit; `55_headline_table.py` writes the pooled headline metrics. `49_second_opinion_brief.py` builds `reports/SECOND_OPINION_BRIEF.md`.

- The final report was edited by hand after Stage C. Re-running 10_finalize.py regenerates it from its template and would

drop the Stage C and Stage E sections; port them first.

5. What was actually used

Element	What was used	Where it is recorded
Dataset	ISIC collection 212 — 11,720 images, of which 10,015 are the labelled HAM10000 cohort	artifacts/logs/data_audit.json
Lesion masks	Released HAM10000 segmentations (X1 tier), 10,015 available	data/manifests/ mask_manifest_x1.csv
Classifier	torchvision ViT-B/16, ImageNet-1k pretrained, 224 px, 196 patches	configs/vit.json
Diffusion model	Upstream DM-AHR (SR3-style) pinned at commit 0aee1bb, vendored in third_party/dmahr	reports/final_research_report.md
Classical baseline	Implemented here: multi-orientation black-hat detection + Telea inpainting	src/dm_ahr_vit/hair/dullrazor.py
Compute	Modal, NVIDIA A100-80GB PCIe; Python 3.11.12, torch 2.5.1+cu124, CUDA 12.4	artifacts/logs/ environment_report.json
Statistics	Lesion-group bootstrap (10,000 reps) and Wilcoxon signed-rank, both Holm-corrected	artifacts/logs/ statistical_comparisons.csv

No clinical claim is made or implied anywhere in the code, figures or reports.

6. Repository map

```
configs/                  frozen parameters, one file per concern
  protocol.json           seeds, split fractions, endpoints, planned comparisons, spend cap
  vit.json                ViT recipe: optimiser, schedule, augmentation, batch size
  hair_screen.json        hair-detector parameters (frozen before test scoring)
```

```
  dullrazor.json         Arm B parameters
  dmahr.json             diffusion settings and pinned upstream commit
  sampling.json          PSNR/SSIM and XAI measurement settings

src/dm_ahr_vit/          the library — runs on CPU, no Modal import anywhere
  config.py              config loading and hashing
  paths.py               DATA_ROOT / ARTIFACT_ROOT resolution for every path
  utils.py               logging, timing, run registry
  data/
    isic.py              ISIC collection download
    ham10000.py          HAM10000 metadata and mask handling
    manifest.py          per-image manifest and near-duplicate link table
    splits.py            group-aware, class-stratified, frozen splits
    masks.py             lesion-mask loading and resizing
    synthetic_hair.py    synthetic hairy/hairless pair generation
    dermahair.py         DERMAHAIR source mapping (blocked: no credentials)
    dataset.py           torch Dataset and transforms for every arm
  hair/
    screening.py         black-hat/top-hat hair-presence detector
    dullrazor.py         Arm B: morphology + Telea inpainting
  diffusion/
    dmahr_adapter.py     adapter around the pinned upstream implementation
    train.py             leakage-controlled diffusion training loop
  vit/
    model.py             ViT-B/16 construction and checkpoint IO
    train.py             training loop, class weights, early stopping
    metrics.py           accuracy, per-class recall/specificity, F1, AUC
  xai/
    vit_xai.py           Grad-CAM on the patch grid, attention roll-out
  stats/
    bootstrap.py         lesion-group bootstrap, Holm, Wilcoxon, Cohen's dz

scripts/                 numbered phases, each a plain CLI (argparse) — Modal is not
required
  00_download_data.py    fetch collection 212
  01_audit_and_splits.py audit, dedup, freeze split
  02_hair_pipeline.py    screening, Arm B, synthetic pairs
  03_dmahr_smoke.py      upstream audit + smoke test
  03b_dmahr_bench.py     sampling throughput benchmark
  03c_batch_sweep.py     batch-size sweep
  04_dmahr_train.py      diffusion training
  05_dmahr_sample.py     Arm C generation (shardable)
  05b_fill_c_passthrough.py  copy hair-absent images into Arm C
  06_train_vit.py        ViT training for one (arm, seed)
  07_eval_vit.py         locked-test evaluation, per-image predictions
  07b_image_quality.py   PSNR/SSIM on synthetic pairs
  08_xai.py              Grad-CAM / attention roll-out vs lesion masks
  09_stats.py            bootstrap + Wilcoxon + Holm table
  10_finalize.py         report, figures, artifact inventory
  11_reproduction_check.py  rerun one configuration and compare
  12_upload_to_drive.py  copy artifacts to Google Drive
  check_acceptance.py    the gate checker

third_party/dmahr/       upstream DM-AHR code, isolated, notices intact
tests/                   CPU unit tests (28 passing)
notebooks/               results walkthrough, Modal reproduction, native local-GPU run
reports/                 the written deliverables
artifacts/               logs, predictions, panels, checkpoints (hashed inventory)
```

Running without Modal

Nothing in `src/` or `scripts/` imports Modal. Each phase is a plain CLI that resolves its paths from two environment variables:

```
export DATA_ROOT=/path/to/data
export ARTIFACT_ROOT=/path/to/artifacts
python scripts/00_download_data.py
python scripts/01_audit_and_splits.py
python scripts/02_hair_pipeline.py
# ... and so on through 10_finalize.py
```

Modal is a scheduling convenience, not a dependency. `notebooks/03_local_gpu_pipeline.ipynb` drives this path end to end on a local or Colab GPU.

Data preparation phases 1–3 (CPU) were verified natively from a fresh download: they reproduced the reference frozen split hash `fda56cf3…`, 7,470 lesion groups, 8,993 hair-present images and 1,022 synthetic pairs without Modal present.

7. The notebooks, cell by cell

Three notebooks ship with the repository. Each is annotated: a markdown cell above every code cell states what that cell does and why it exists. This section is the written index.

Notebook	Purpose	Needs
01_results_walkthrough.ipynb	reads the committed artifacts and explains the results	CPU only, no Modal, no download
02_pipeline_reproduction.ipynb	runs every phase on Modal	Modal token
03_local_gpu_pipeline.ipynb	runs every phase natively on a local or Colab GPU	a CUDA GPU, no Modal

01_results_walkthrough.ipynb — presenting the study

Cell	What it does
Setup	resolves the artifact directories and defines JSON-loading helpers
Cohort accounting	rebuilds the headline counts from the audit artifact
Split sizes	prints per-class split counts, showing the stratification worked
Leakage check	independently recomputes that no lesion group spans two splits
Hair screening	reports how many images were marked hair-present
Arm definitions	a reference table of what each arm is
Classification results	macro-F1, accuracy, balanced accuracy and AUC per arm
Result figures	the same numbers as bars with error bars
Per-class detail	where the arms differ by diagnosis
Image quality	PSNR/SSIM against hairless references, with the hairy input as a floor
Explanation metrics	overlap and pointing per arm with both chance baselines
Explanation figure	saliency against the uniform-chance line
Comparison table	every endpoint with both tests and Holm correction
Significance summary	the decision a reader cares about, per comparison
Hypothesis verdicts	maps H1/H2/H3 onto the comparisons that bear on them
Visual panels	the contact sheets written during processing
Acceptance gates	the project's own automated checks

03_local_gpu_pipeline.ipynb — running it yourself

Cell	What it does
Environment check	reports the GPU and torch build before anything assumes one

Cell	What it does
Dependencies	installs the pinned versions the results were produced with
Get the code	clones the repository and enters it
Where the data lives	sets DATA_ROOT and ARTIFACT_ROOT; in Colab these point at Drive
Script runner	a wrapper that invokes each phase with those variables set
Download + verify	fetches all 11,720 images and masks, then reads the report back
Audit + split, and verify	rebuilds the frozen split and confirms the counts and leakage check
Hair pipeline	screening, Arm B, Arm A∗ and the synthetic pairs
Released checkpoint	fetches the DM-AHR authors' weights, needed for the smoke test
Smoke test	confirms the vendored upstream code runs and a batch round-trips deterministically
Benchmark	measures sampling throughput, to size the Arm C run
Diffusion sanity + training	overfits a few pairs, then trains on train/val sources only
Arm C, one shard	generates one eighth of the images and reports the timing
Arm C, remaining shards	finishes the rest; safe to re-run because finished images are skipped
Arm C, held-out pairs	generates the pairs that make PSNR/SSIM measurable
Passthrough	copies hair-absent images into Arm C, where they are identical
Arm inventory	counts what exists on disk before training
One ViT run, then all	confirms the recipe works, then trains the remaining eleven
Evaluation	scores each frozen checkpoint once on the test set
Image quality, XAI	PSNR/SSIM, then Grad-CAM and attention roll-out with baselines
Statistics, report, gates	the comparison table, generated report, inventory and acceptance check
Outputs	counts the files produced, and how to explore them

`02_pipeline_reproduction.ipynb` follows the same phases, but each cell shells out to `modal run`. It needs no local GPU; the runtime selected in Colab is irrelevant to it.

8. Where each reported number comes from

Report claim	Source artifact	Field
11,720 collection images	data_audit.json	collection_total
10,015 confirmatory, 1,705 quarantined	data_audit.json	c0_decoded, c0_extra
7,470 lesion groups	data_audit.json	groups_total
7,011 / 1,502 / 1,502 split	data_audit.json, splits.csv	frozen_splits.counts

Report claim	Source artifact	Field
Split hash fda56cf3...	split_summary.json	frozen hash
8,993 hair-present (89.8%)	hair_screening.csv	hair_present
186 held-out pairs	paired_psnr_ssim_test.csv	row count
Macro-F1 A 0.750 / B 0.698 / C 0.426	classification_seed_summary.csv	macro_f1_mean, _sd
Per-class recall/specificity/AUC	classification_per_class.csv	recall_mean, specificity_mean, auc_mean
PSNR 35.732 / 16.312 / 22.127	statistical_comparisons.csv	arm_a_mean, arm_b_mean
Grad-CAM and roll-out overlap	xai_localization_*.csv, statistical_comparisons.csv	*_overlap20
p-values, CIs, effect sizes	statistical_comparisons.csv	p_raw, p_holm, wilcoxon_p_*, effect_size_dz
GPU and versions	environment_report.json	device, torch, cuda, python
Gate status	acceptance_report.json	gates[], summary

9. Design decisions, and the reason for each

Decision	Reason
Quarantine 1,705 extra images	The collection is not the paper's cohort. Mixing them without justification would silently change the study.
Split by lesion, not image	Multiple photographs of one lesion across splits would leak identity.
Pixel-confirm near-duplicates	70,062 dHash candidates were mostly background collisions; only 21 survived pixel checks.
Freeze the split before training	A split chosen after seeing results cannot be trusted.
Freeze the hair threshold before test scoring	Otherwise the detector is tuned on the test set.
Train the diffusion model ourselves	The released weights have undocumented training data, so leakage cannot be excluded.
Add Arm A∗	Arm C changes resolution (128→224 px). Without A∗, resolution and diffusion artefacts are confounded.
Score the hairy input as a floor	Makes "worse than doing nothing" visible instead of implied.
Per-image predictions, not just means	Every statistic becomes recomputable and auditable.
Two statistical tests	The proposal specifies Wilcoxon beside bootstrap CIs; they answer slightly different questions.
Chance baselines for XAI	Mask size varies; overlap without a baseline is uninterpretable.
Report a labelled 1,000-step ablation	A complete four-arm comparison at reduced budget is more informative than a missing arm, provided the deviation is stated plainly.

10. Deviations from the proposal

Every one is recorded in `reports/decision_log.md` with its reason. Summary:

#	Deviation	Status
D1	Diffusion trained 20,000 iterations, not 660,000, with a defective script	Documented, limits Arm C
D9	DERMAHAIR pairs unavailable (Kaggle-gated); synthetic hair used	Documented, F04 PENDING
D12	Arm C sampled at 1,000 steps not 2,000	Labelled ablation
D14	Implementation on Modal, not Google Colab as the proposal states	Documented
D15	Augmentation ±15° rotation, not ±90° with explicit zoom/shift	Documented, matched across arms
D16	Class weights only, no class-balanced sampling	Documented

Two further points belong in the same list but are not deviations from the code — they are properties of the data. The hair-screening threshold is not verified against real hair annotations, because none exist in this dataset. And the lesion masks are the released HAM10000 segmentations, which the source does not claim were dermatologist-traced.

11. Glossary

Term	Meaning here
Macro-F1	F1 computed per class then averaged, so rare classes count as much as nv. The primary endpoint.
Balanced accuracy	Mean of per-class recall; another imbalance-robust summary.
Lesion group	All images of one lesion, including pixel-confirmed near-duplicates. The unit of splitting and of statistics.
Arm	One input condition: A, B, C or A∗.
Ablation	A deliberately reduced run, labelled as such, used to answer "does a cheaper version help". Arm C at 1,000 steps is one.
PSNR / SSIM	Pixel-fidelity and structural-similarity measures; both need a hairless reference image.
Grad-CAM	Gradient-weighted saliency over the ViT patch grid.
Attention roll-out	Attention matrices multiplied across layers to give token-to-token flow.
Pointing game	Does the single most salient pixel land inside the lesion mask?
Bootstrap CI	Confidence interval from resampling lesion groups 10,000 times.
Holm correction	A step-down method controlling family-wise error across multiple comparisons.
Cohen's dz	Standardised mean difference for paired data.

12. Questions a supervisor is likely to ask

"Why is Arm C so much worse? Isn't that just a broken implementation?"

Partly a budget problem, and partly a defect in our training script. The report says so first: the script gave the network an inverted noise-level signal (found after the primary results), the run used 20,000 training iterations instead of 660,000, 1,000 sampling steps instead of 2,000, and 128 px upsampled to 224. The smoke test, the geometry check, the deterministic reload and the panelled outputs show the sampling implementation runs correctly. The report labels Arm C a failed replication and does not present it as the method's published performance.

"How do you know there is no leakage?"

Three independent guards: the split is grouped by lesion plus pixel-confirmed duplicates; the zero-overlap property is recomputed from `splits.csv` and enforced by an automated test; and the diffusion model and hair threshold were fit on train/validation only before the test set was scored.

"Why not report accuracy?"

Because `nv` is roughly two thirds of the data, accuracy rewards ignoring melanoma. Macro-F1 is primary, with balanced accuracy, per-class recall and specificity, and AUC alongside.

"Three seeds is thin."

Correct, and that is why inference is over ~1,437 lesion groups with 1,502 test images, not over three seed means. Seeds are averaged inside each bootstrap replicate and never counted as independent samples.

"Is the diffusion model really DM-AHR?"

Yes — the upstream code is vendored at a pinned commit and its released checkpoint is loaded for the smoke test. It is DM-AHR at a fraction of its training and sampling budget, which is a different claim from "DM-AHR does not work", and the report keeps that distinction explicit.

"What would it cost to do properly?"

Arm C sampling alone is about 5.4 A100-hours at 1,000 steps and 10.9 at the 2,000-step reference, before diffusion training (20,000 iterations) and twelve ViT runs. `reports/compute_plan.md` has the measured per-unit costs.

13. What is deliberately not claimed

- That diffusion hair removal helps or harms classification in general. The tested model was

a failed replication.

- That classical hair removal hurts classification. Our frozen DullRazor settings damage lesions

on real hair, so the original-against-DullRazor gap is confounded.

- That any arm cleanly removed real hair. The released diffusion model left real hair in place.
- That the explanations show clinically correct reasoning. They show spatial alignment with released

masks.

- That the hair detector is validated on real hair. No independent annotations exist.
- That the results transfer to any other dataset, scanner or population.
- Any clinical, diagnostic or patient-facing use.
- Any resolution of the licence question. HAM10000 is CC BY-NC 4.0; the DM-AHR repository's README

says CC BY-NC 4.0 while its LICENSE file says Apache-2.0. Commercial use is unresolved and needs a rights decision.